

Lane Keeping

Lane Keeping

1. Course Content
 2. Run Example Program
 3. Source Code Analysis
- 3.1, auto_drive.py

1. Course Content

Note: This program runs on a sandbox map; you will need to adjust the program for your personal map.

Start the autonomous driving program and use lane keeping centered autonomous driving function on the sandbox map

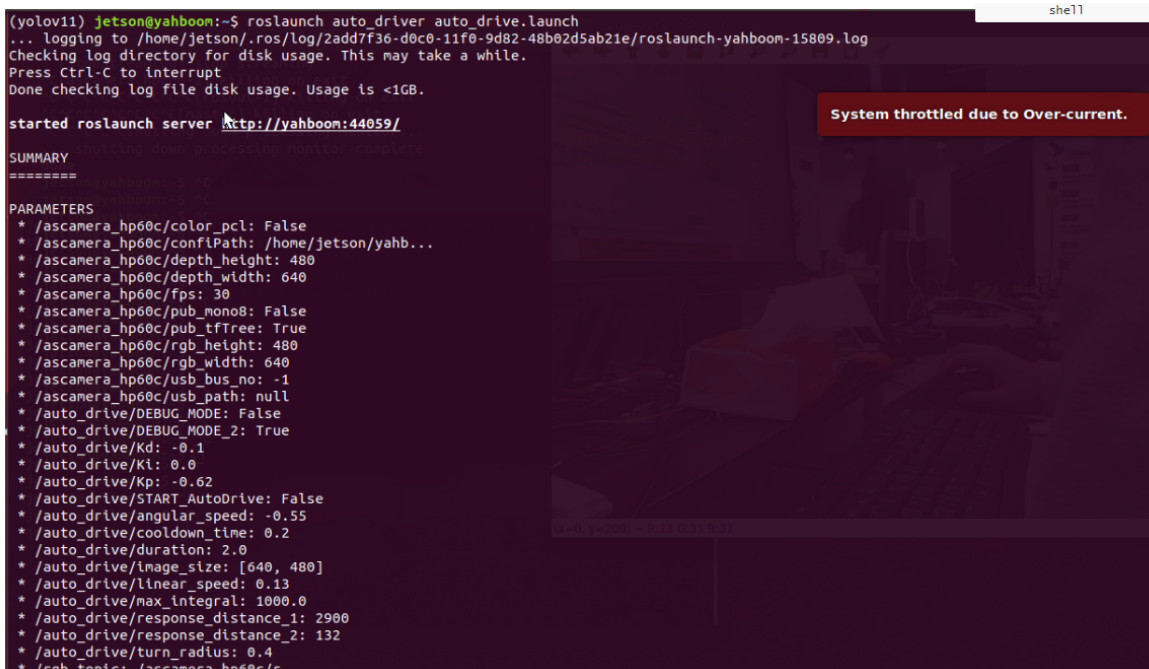
2. Run Example Program

Autonomous driving source code path:

```
/home/jetson/yahboomcar_auto/src/auto_driver/scripts/auto_drive.py
```

- Start camera + chassis + autonomous driving node (need to run in virtual environment terminal)

```
roslaunch auto_driver auto_drive.launch
```



```
(yolov11) jetson@yahboom:~$ roslaunch auto_driver auto_drive.launch
... logging to /home/jetson/.ros/log/2add7f36-d0c0-11f0-9d82-48b02d5ab21e/roslaunch-yahboom-15809.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://yahboom:44059/

SUMMARY
=====
PARAMETERS
* /ascamera_hp60c/color_pcl: False
* /ascamera_hp60c/confiPath: /home/jetson/yahb...
* /ascamera_hp60c/depth_height: 480
* /ascamera_hp60c/depth_width: 640
* /ascamera_hp60c/fps: 30
* /ascamera_hp60c/pub_mono8: False
* /ascamera_hp60c/pub_tfTree: True
* /ascamera_hp60c/rgb_height: 480
* /ascamera_hp60c/rgb_width: 640
* /ascamera_hp60c/usb_bus_no: -1
* /ascamera_hp60c/usb_path: null
* /auto_drive/DEBUG_MODE: False
* /auto_drive/DEBUG_MODE_2: True
* /auto_drive/Kd: -0.1
* /auto_drive/Kl: 0.0
* /auto_drive/Kp: -0.62
* /auto_drive/START_AutoDrive: False
* /auto_drive/angular_speed: -0.55
* /auto_drive/cooldown_time: 0.2
* /auto_drive/duration: 2.0
* /auto_drive/image_size: [640, 480]
* /auto_drive/linear_speed: 0.13
* /auto_drive/max_integral: 1000.0
* /auto_drive/response_distance_1: 2900
* /auto_drive/response_distance_2: 132
* /auto_drive/turn_radius: 0.4
* /ascamera_hp60c/usb_bus_no: -1
* /ascamera_hp60c/usb_path: null
```

After successful startup, the display will not move to test road sign recognition. (If you want it to move, change the parameter START_AutoDrive in auto_drive_node.yaml to True)

- Parameter Configuration

Parameter path:

```
/home/jetson/yahboomcar_auto/src/auto_driver/config/auto_drive_node.yaml
```

```
turn_radius: 0.4
linear_speed: 0.13
angular_speed: -0.55
duration: 2.0
response_distance_1: 2900
response_distance_2: 132
cooldown_time: 0.2
DEBUG_MODE: True
DEBUG_MODE_2: False
START_AutoDrive: True
Kp: -0.62
Ki: 0.0
Kd: -0.1
max_integral: 1000.0
image_size: [640, 480]
```

【turn_radius】 : Turning radius. Normally no need to change this parameter

【drive_speed】 : Linear speed magnitude. The driving speed of the car, factory default is 0.13. If you modify this parameter, you need to debug other parameters accordingly

【angular_speed】 : Turning angular speed magnitude

【duration】 : Right turn time when recognizing right turn sign

【cooldown_time】 : Road sign recognition cooldown time

【DEBUG_MODE】 : Lane line visualization debug parameter, enabling may affect centering effect

【DEBUG_MODE_2】 : Road sign visualization debug parameter

【START_AutoDrive】 : Control car movement parameter, true means the car can move

【kp、ki、kd】 : Car centering PID parameters

The above parameters are the main adjustment and optimization parameters. Keep other parameters at factory default values. After modifying parameters, restart the launch file to use the modified parameters

- Centerline Adjustment

If the car turns too early or too late during lane keeping, you can debug the centerline to adjust the car's turning timing

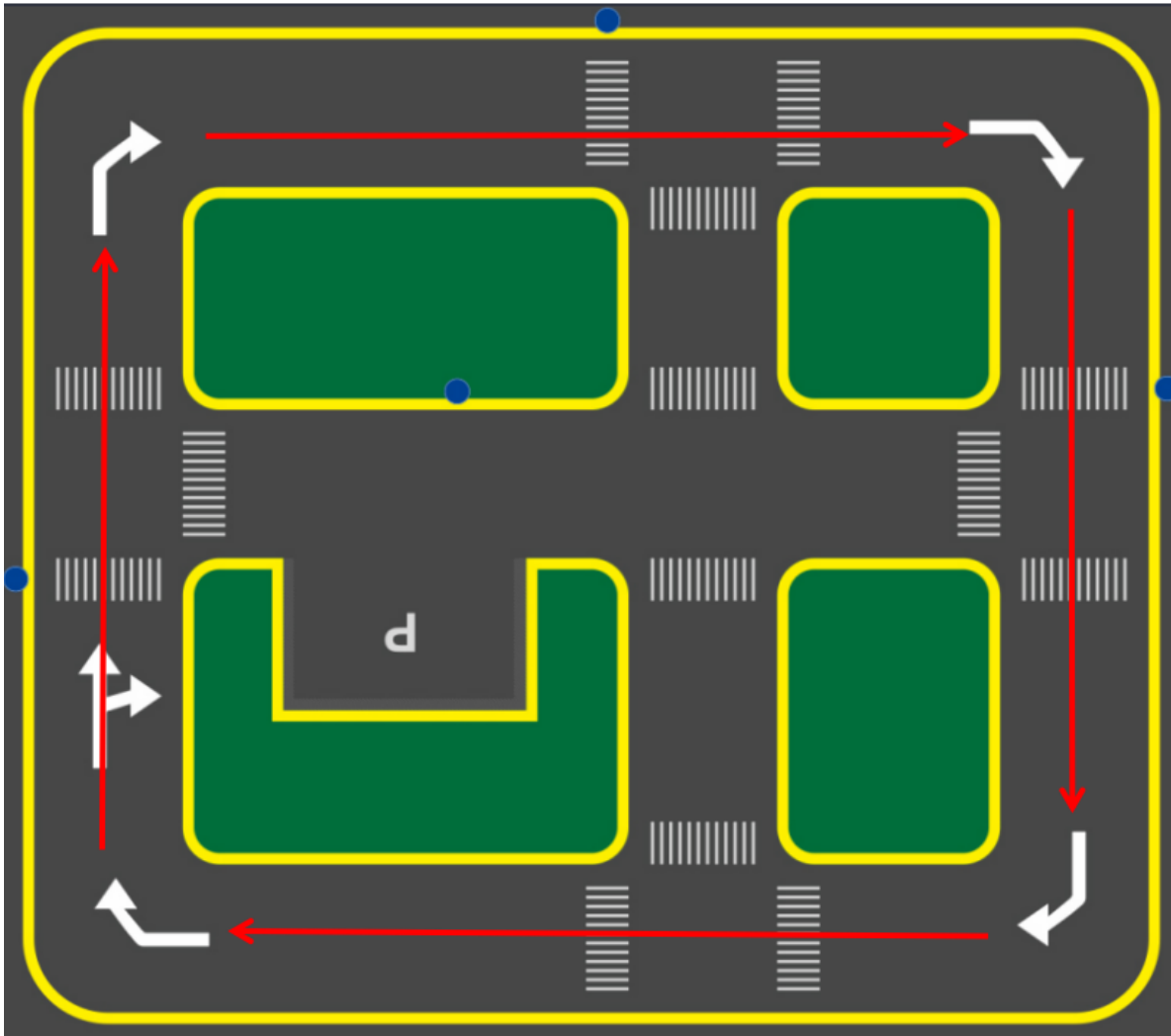
Parameter path:

/home/jetson/yahboomcar_auto/src/auto_driver/config/midline_points.yaml

```
midline_points:
  start_point:
    x: 360
    y: 477
  end_point:
    x: 355
    y: 424
  midline_length: 120
  mid_distance: [200,40]
```

Adjust the car's turning timing by modifying the `midline_length` parameter. Taking 120 length as an example, if the turn is too early, you can try changing 120 to 115 for debugging. You can also debug by slightly adjusting the camera angle downward, which will also make the turn slightly later. Debug according to the actual environment.

- The car will then perform autonomous driving along the outer lane
 - On straight road sections, it will automatically execute straight driving and left-right lane centering
 - On curved road sections, it will automatically turn and return to lane keeping mode after exiting the curve



3. Source Code Analysis

3.1, auto_drive.py

Program source code path

```
/home/jetson/yahboomcar_auto/src/auto_driver/scripts/auto_drive.py
```

Control Flow is divided into

1. Receive and process image data
2. Detect lane lines and traffic signs
3. Decide control strategy based on detection results
4. Publish control commands (speed, direction)
5. Loop through the above steps

```

def _handle_lane_detect_result(self, left_box, right_box, frame):
    '''Handle lane detection results and control vehicle'''
    error_angular = [0.0, 0.0]
    if self.flag != False:
        return frame
    # Classify and process based on detected lane line conditions
    if left_box is not None and right_box is not None and left_box[0] <
right_box[0]:
        # Both sides have lane lines
        self.error_mode = 0
        frame = self.lane_detect.draw_left_line(frame, left_box)
        frame = self.lane_detect.draw_right_line(frame, right_box)
        frame, x_center, y_center =
self.lane_detect.draw_center_point(frame, left_box, right_box)
        error_angular = self.lane_detect.cal_error_angular(x_center,
y_center)

        # PID control
        if abs(error_angular[1]) >= 0.03 and abs(error_angular[1]) < 0.3:
            angular_z = self.pid_controller.pid_control(error_angular[1])
            angular_z = max(min(angular_z, 0.3), -0.3)
            self._set_current_speed(linear_x=self.drive_speed,
angular_z=angular_z)
        else:
            self._set_current_speed(linear_x=self.drive_speed,
angular_z=0.0)

    elif left_box is not None and right_box is None and left_box[0] < 320:
        # Only left lane line
        self.error_mode = 1
        self.lane_detect.prev_center = None
        frame = self.lane_detect.draw_left_line(frame, left_box)

        if_infer = self.lane_detect.IF_lane_intersection(left_box)
        if if_infer is not None:
            self._set_current_speed(linear_x=self.drive_speed,
angular_z=self.angular_speed)
        else:
            self._set_current_speed(linear_x=self.drive_speed,
angular_z=0.0)

    elif right_box is not None and left_box is None:
        # Only right lane line
        self.error_mode = 2
        self.lane_detect.prev_center = None
        frame = self.lane_detect.draw_right_line(frame, right_box)
        self._set_current_speed(linear_x=self.drive_speed, angular_z=0.0)
    else:
        # No lane lines detected
        self.error_mode = 99
        self.lane_detect.prev_center = None
        self._set_current_speed(linear_x=self.drive_speed, angular_z=0.0)

    # Display angle deviation
    cv2.putText(frame, "Angle Dev: {:.2f} rad".format(error_angular[1]),
(10, 105),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 0, 255), 2)
    frame = self.lane_detect.draw_midline(frame)

```

```
return frame
```

The above is the key lane line centering program, which judges the turning timing and centering walking timing based on the angle between the left and right yellow lane lines and the centerline.